

# IHK-Leitfaden – Teil 3: OOP, Tests & Patterns (FIAE, AP2) – Druckfreundlich

Nur Deutsch · Java-ähnlicher Pseudocode · ohne dunkle Hintergründe

**Prüfungstipp:** In Teil 3 prüft die IHK oft: saubere Klassen (Attribute, Konstruktoren, Methoden), einfache Tests (Unit, Black/White-Box) und Grund-Patterns (MVC, Singleton, Observer).

## 1. Objektorientierung (OOP)

### Klasse + Konstruktor + Getter

```
class Jahresstatistik {  
    private int minVerbrauch;  
    private int maxVerbrauch;  
  
    Jahresstatistik(int min, int max) {  
        this.minVerbrauch = min;  
        this.maxVerbrauch = max;  
    }  
  
    int getMinVerbrauch() { return minVerbrauch; }  
    int getMaxVerbrauch() { return maxVerbrauch; }  
}
```

### Funktion mit Objektrückgabe

```
Jahresstatistik ermittleJahresstatistik(int[][] verbrauch) {  
    int min = Integer.MAX_VALUE;  
    int max = Integer.MIN_VALUE;  
    for (int m = 0; m < verbrauch.length; m++) {  
        int sum = 0;  
        for (int t = 0; t < verbrauch[m].length; t++) sum += verbrauch[m][t];  
        if (sum < min) min = sum;  
    }  
    return new Jahresstatistik(min, max);  
}
```

```
        if (sum > max) max = sum;
    }
    return new Jahresstatistik(min, max);
```

**Tipp:** Objektrückgabe = mehrere Werte bündeln. Kapselung (private Felder, Getter) kurz erwähnen.

## 2. Tests (IHK-typisch)



### Black-Box vs. White-Box

- **Black-Box:** Test nach Spezifikation (Eingabe/Erwartung). Code ist egal.
- **White-Box:** Test nach Code-Struktur (Zweige, Bedingungen, Pfade).

### Unit-Test (Beispiel)

```
// Testfall: min/max korrekt?
int[][] v = { {10,20,30}, {15,15,15}, {40,20,10} };
Jahresstatistik js = ermittleJahresstatistik(v);
assert(js.getMinVerbrauch() == 45);
assert(js.getMaxVerbrauch() == 70);
```

### Regressionstest

Stellt sicher, dass neue Änderungen keine bereits funktionierende Funktionalität kaputt machen. Immer nach Refactorings oder Releases laufen lassen.

**Tipp:** Nenne bei Tests immer: *Zweck, Testdaten, Erwartung*. Für White-Box: Branch-Abdeckung erwähnen.

## 3. Design Patterns (Kernwissen)

### MVC – Model View Controller

- **Model:** Daten & Logik (z. B. Jahresstatistik)
- **View:** Darstellung (UI/Output)
- **Controller:** verbindet Eingaben mit Model-Operationen

```
// Mini-Skizze (Pseudocode)
class Model { /* Daten & Logik */ }
class View { void show(Model m) { /* Ausgabe */ } }
class Controller {
    Model m; View v;
    void handleInput(...) { /* m verändern, v.show(m) */ }
}
```

### Singleton

Genau eine Instanz im System (z. B. Config, Logger).

```
class Logger {
    private static Logger instance;
    private Logger() {}
    static Logger getInstance() {
        if (instance == null) instance = new Logger();
        return instance;
    }
}
```

*Vorteil:* zentrale Instanz. *Nachteil:* kann Tests erschweren.

### Observer

Mehrere *Observers* reagieren auf Änderungen eines *Subjects* (Publish/Subscribe).

```
interface Observer { void update(int value); }  
class Subject {  
    List<Observer> obs = new ArrayList<>();  
    void attach(Observer o){ obs.add(o); }  
    void setValue(int v){ /* ... */ notifyAll(v); }  
    void notifyAll(int v){ for (Observer o: obs) o.update(v); }  
}
```

*Nutzung:* UI aktualisieren, wenn sich Daten ändern.

**Tipp:** In Antworten kurz Zweck, Aufbau und typisches Einsatzszenario nennen. 1–2 Vor-/Nachteile reichen.

## 4. Debugging – typische Schritte

- Fehler reproduzieren (Eingaben, Umgebung notieren).
- Log-Ausgaben oder Breakpoints setzen.
- Hypothese → Stelle im Code eingrenzen → prüfen.
- Fix implementieren → **Regressionstest** laufen lassen.
- Ursache dokumentieren (Lessons Learned).

### Mini-Beispiel

```
// Bug: Prüfziffer falsch berechnet (Rückgabe war sum % 10)  
int p = (10 - (sum % 10)) % 10; // fix: Differenz zum nächsten Zehner
```

## 5. Spickzettel

| Thema          | Kernpunkte                                                         |
|----------------|--------------------------------------------------------------------|
| OOP            | Klasse, Attribute, Konstruktor, Methoden, Getter/Setter, Kapselung |
| Objektrückgabe | <code>return new Klasse(w1, w2);</code> – mehrere Werte bündeln    |
| Unit-Test      | kleinste Funktion testen; <code>assert(Erwartung)</code>           |
| Black-Box      | gegen Spezifikation (Input/Output), Code egal                      |

| Thema      | Kernpunkte                                                       |
|------------|------------------------------------------------------------------|
| White-Box  | Zweig-/Pfadabdeckung; Tests aus Code abgeleitet                  |
| Regression | stellt sicher: Alte Funktionen bleiben intakt                    |
| MVC        | Trenne Daten (Model), Darstellung (View), Steuerung (Controller) |
| Singleton  | eine Instanz; Zugriff über <code>getInstance()</code>            |
| Observer   | Subject benachrichtigt Beobachter bei Änderung                   |
| Debugging  | Reproduzieren → Eingrenzen → Fix → Regressionstest               |

## 6. Checkliste – Was du für Teil 2 können musst

- Pseudocode sicher schreiben (Java-ähnlich, klar strukturiert)
- Zählen, Summieren, Durchschnitt (Array)
- Min/Max (1D) und 2D-Array (verschachtelte Schleifen)
- Objektrückgabe (z. B. Jahresstatistik)
- Sortieren: BubbleSort, InsertionSort ( $O(n^2)$ )
- Suchen: Linear ( $O(n)$ ), Binär ( $O(\log n)$ , nur sortiert)
- Testarten: Black-Box, White-Box, Unit, Regression
- Patterns: MVC, Singleton, Observer (Zweck & Skizze)
- Debugging-Grundschrirte; kleine Fixes im Code

© Lernhilfe – „Teil 3: OOP, Tests & Patterns“ – Druckfreundliche Version. Zum PDF-Export: Browser → Drucken  
→ Als PDF speichern.